

Understanding APIs

Section A — API Fundamentals

Q1. What does API stand for, and what is its primary purpose in software communication?

API stands for Application Programming Interface. In simple terms, it is a set of rules that lets two pieces of software talk to each other without either one needing to know how the other is built internally. Its main purpose is to let an application request data or services from another application (or a server) in a predictable, standardized way, so developers can reuse existing functionality instead of building everything from scratch.

Q2. Using the restaurant analogy from the session, explain the role of the "waiter" in an API request.

In the restaurant analogy, the customer is the client, the kitchen is the server, and the API is the waiter who moves between them. The customer never walks into the kitchen to tell the chef what to cook — they tell the waiter, the waiter carries that order to the kitchen in a format the kitchen understands, and once the food is ready, the waiter brings it back to the table. The API plays exactly this middleman role: it carries the request from the client to the server and carries the response back, without the client needing to know anything about how the kitchen (server) actually works.

Q3. Give two real-world examples of applications that rely on APIs (other than those covered in the session) and name the type of API each likely uses.

First, a food delivery app like Swiggy or Zomato uses a mapping API (similar to Google Maps API) to show delivery routes and live rider location — this would be a REST API returning JSON. Second, a travel booking site like MakeMyTrip uses payment gateway APIs (such as Razorpay or Stripe) to process ticket payments securely — again typically a REST-based API that communicates over HTTPS using JSON requests and responses.

Q4. Why do companies prefer using existing APIs (for maps, payments, login, etc.) instead of building these services from scratch?

Building a mapping system, a payment processor, or a login/authentication system from the ground up is expensive, time-consuming, and carries a lot of risk — especially around security and compliance. By using an established API, a company gets a tested, reliable, and continuously maintained service instantly, and can focus its own engineering effort on its core product instead of reinventing infrastructure that already exists. It also reduces long-term maintenance overhead since the API provider is responsible for keeping the service updated and secure.

Section B — Client–Server Architecture & Data Travel

Q5. Differentiate between the roles of a "Client" and a "Server" in client-server architecture, with one example of each.

A client is the application or device that initiates a request for information or a service — for example, a web browser like Chrome requesting a webpage. A server is the machine or program that receives that request, processes it, and sends back a response — for example, the servers that host a website like amazon.com and respond with product data whenever a browser asks for it. The client always starts the conversation; the server listens and responds.

Q6. What is the purpose of DNS, and why can't computers communicate using domain names alone?

Computers on the internet find each other using IP addresses, not names, so DNS (Domain Name System) exists to translate a human-friendly domain name like www.google.com into the numeric IP address the underlying network actually uses to route traffic. Domain names alone can't be used for communication because the internet's routing protocols work at the IP address level — DNS is simply the lookup step that happens first, quietly in the background, before the actual connection is made.

Q7. Briefly explain how data physically travels across the internet in packets before reaching its destination.

Before data is sent, it is broken down into small chunks called packets, each carrying a piece of the original data along with metadata such as the destination address and sequence number. These packets travel independently across a chain of routers, sometimes taking different paths, until they reach the destination device. Once there, the destination reassembles the packets in the correct order using the sequence information, recreating the original message — whether that's a webpage, an image, or an API response.

Section C — HTTP & Request–Response Cycle

Q8. Define HTTP and explain why it is described as a "protocol."

HTTP stands for Hypertext Transfer Protocol. It is the set of rules that governs how a client and a server exchange messages over the web — how a request should be formatted, what a valid response looks like, and what different status outcomes mean. It is called a protocol because, just like a language shared by two people, both the client and the server must "speak" HTTP in the same structured way for the communication to be understood correctly.

Q9. List the five components that make up an HTTP request.

- URL (or endpoint) — the address of the resource being requested
- Method — the action being requested, such as GET or POST
- Headers — metadata about the request, such as content type or authentication
- Body — the actual data being sent, usually for POST/PUT/PATCH requests
- Query parameters (or path parameters) — extra details attached to filter or refine the request

Q10. What is the difference between `response.status_code` and `response.json()` when reading an API response?

`response.status_code` gives a short numeric code (like 200 or 404) that tells you whether the request succeeded, failed, or ran into some other outcome, without revealing any of the actual data. `response.json()` goes a step further and converts the response body — assuming it's in JSON format — into a Python dictionary (or list) so the actual content returned by the server, such as user details or search results, can be used in code. In short, `status_code` tells you what happened, while `json()` gives you what was returned.

Q11. Using the pizza-ordering analogy, match each part of an order (URL, Method, Headers, Body, Authentication) to its real-world equivalent.

Order Component	Real-World Equivalent
URL	The pizza shop's address — where the order is being sent
Method	What you want to do — place a new order (POST), check order status (GET), or cancel it (DELETE)
Headers	Extra instructions given at the counter, like "no onions" or your membership card number
Body	The actual order details — size, toppings, quantity
Authentication	Showing your loyalty card or membership ID to prove you are a valid, recognized customer

Section D — URLs, Endpoints & HTTP Methods

Q12. Break down the URL `https://api.openai.com/v1/chat/completions` into its four components.

Component	Value
Protocol	<code>https://</code>
Domain / Host	<code>api.openai.com</code>
Path / Version	<code>/v1</code>
Endpoint	<code>/chat/completions</code>

Q13. What is an API endpoint? Give an example of two different endpoints for a shopping application.

An endpoint is a specific URL path on an API that represents one particular resource or action a client can access — think of it as one specific "counter" within a larger service. For a shopping application, one endpoint might be `/products`, which returns the catalog of items available, while another might be `/orders/{order_id}`, which returns the details of one specific customer order.

Q14. Match each HTTP method — GET, POST, PUT, PATCH, DELETE — to its correct purpose.

Method	Purpose
GET	Retrieve existing data without changing anything on the server
POST	Create a new resource on the server
PUT	Replace an existing resource entirely with new data
PATCH	Update part of an existing resource without replacing the whole thing
DELETE	Remove an existing resource from the server

Q15. Explain the key difference between PUT and PATCH when updating a resource.

PUT replaces the entire resource with the new data you send — if you leave out a field, that field is typically wiped out or reset. PATCH, on the other hand, only updates the specific fields you include in the request and leaves everything else untouched. So if a user record has a name, email, and phone number, and you only want to update the phone number, PATCH is the safer and more efficient choice, while PUT would require you to resend the entire record.

Section E — Status Codes, Headers & Authentication

Q16. What do the status codes 401 and 403 indicate, and how are they different from each other?

A 401 (Unauthorized) means the request lacks valid authentication credentials altogether — essentially, the server doesn't know who you are, so log in or provide a valid API key. A 403 (Forbidden) means the server does know who you are, but you don't have permission to access that specific resource. In short, 401 is a login/identity problem, while 403 is a permissions problem for an already-identified user.

Q17. Why is a 429 status code returned, and what should a well-designed application do when it receives one?

A 429 (Too Many Requests) is returned when a client has exceeded the rate limit the API allows within a given time window — the server is protecting itself from being overwhelmed. A well-designed application should not keep hammering the API immediately after receiving this code; instead, it should back off and retry after a delay, ideally using an exponential backoff strategy, and respect any "Retry-After" header the server provides.

Q18. What is the purpose of the "Authorization: Bearer <API_KEY>" header?

This header is how a client proves its identity to the server on every request. The word "Bearer" indicates the token type, and the API key itself acts like a temporary access pass — whoever presents ("bears") that key is trusted to be an authorized caller. The server checks this header before processing the request, and rejects it with a 401 if the key is missing or invalid.

Q19. List three golden rules for handling API keys securely, as covered in the session.

- Never hard-code API keys directly in source code — store them in environment variables or a secrets manager instead.
- Never commit API keys to a public (or even private) GitHub repository — use a .gitignore entry for any file that holds them.
- Rotate and revoke keys periodically, and immediately if a key is ever suspected of being exposed or leaked.

Section F — JSON, the Language of APIs

Q20. Why is JSON considered the "standard format" for API responses?

JSON (JavaScript Object Notation) is lightweight, human-readable, and maps naturally onto data structures like dictionaries and lists that almost every modern programming language already supports. Because it's language-independent and easy to parse, it has become the common format that both the client and the server can rely on, regardless of what programming language either side is written in.

Q21. Explain one key difference between a Python dictionary and JSON.

A Python dictionary is a native data structure that only exists inside a running Python program, and it can hold Python-specific values like True/False, None, or even custom objects. JSON is a plain text format used to exchange data between systems, and it does not understand Python-specific types — for instance, JSON uses lowercase true/false/null instead of Python's True/False/None. So a dictionary needs to be converted (serialized) into a JSON string before it can be sent over the network, and JSON needs to be parsed back into a dictionary once received.

Q22. Convert the following Python dictionary into valid JSON format: {"name": "Employee", "department": "IT", "active": True}

```
{
  "name": "Employee",
  "department": "IT",
  "active": true
}
```

Note that the only real change needed is that Python's True becomes the lowercase true in JSON, since JSON does not use Python's capitalized boolean keywords.

Section G — The Python requests Library

Q23. Which function from the requests library would you use to (a) read data, (b) create new data, and (c) delete data?

- (a) Read data → requests.get()
- (b) Create new data → requests.post()
- (c) Delete data → requests.delete()

Q24. What is the purpose of the timeout parameter in a requests.get() call, and why is it considered a best practice?

The timeout parameter tells the requests library how many seconds to wait for a server to respond before giving up and raising an exception. It is considered a best practice because, without it, a request can hang indefinitely if the server is slow or unreachable, which can freeze the entire application. Setting a reasonable timeout ensures the program fails fast and can handle the error gracefully instead of getting stuck.

Q25. Write a short Python try/except block that safely calls an API and handles a `requests.exceptions.RequestException`.

```
import requests

try:
    response = requests.get(
        "https://api.example.com/data",
        timeout=5
    )
    response.raise_for_status()
    data = response.json()
    print(data)
except requests.exceptions.RequestException as e:
    print(f"API call failed: {e}")
```

Q26. What is the advantage of using a `requests.Session()` object over calling `requests.get()` repeatedly?

A Session object reuses the same underlying TCP connection across multiple requests instead of opening and closing a new connection every single time, which makes repeated calls to the same host noticeably faster. It also lets you set things like headers, authentication, or cookies once on the session, and have them automatically applied to every request made through that session — which is both more efficient and less repetitive than passing the same parameters to `requests.get()` every time.

Section H — Real-World Application & Best Practices

Q27. Describe, step by step, how a prompt typed into an AI chatbot travels to the AI model and back as a response.

- The user types a prompt into the chatbot's interface (the client).
- The client packages the prompt into an HTTP POST request, placing the prompt text inside the request body as JSON, and attaches an Authorization header with the API key.
- This request travels over the internet, broken into packets, routed through DNS-resolved servers, until it reaches the AI provider's API server.
- The server authenticates the request, passes the prompt to the underlying AI model, and waits for the model to generate a response.
- The server packages the model's output back into a JSON response and sends it back to the client, along with a status code such as 200 OK.
- The client reads `response.json()`, extracts the generated text, and displays it to the user in the chat window.

Q28. List four best practices an employee should follow when working with any company API in a production environment.

- Always set a timeout on every request so the application never hangs waiting on a slow server.
- Wrap API calls in try/except blocks to handle failures gracefully instead of letting the program crash.
- Never expose or hard-code API keys in source code — use environment variables or a secure secrets manager.
- Respect rate limits by checking for 429 responses and implementing retry logic with backoff, rather than repeatedly hammering the API.

Q29. A colleague hard-codes an API key directly into a Python script and pushes it to GitHub. Identify the risk and explain the correct approach.

The risk is that the API key is now publicly visible in the repository's commit history, even if it's later removed from the file — anyone who finds it can use it to make requests under the company's account, potentially running up costs, accessing private data, or exceeding usage limits before anyone notices. The correct approach is to store the key in an environment variable (or a secrets manager) and load it into the script at runtime using something like `os.environ.get("API_KEY")`, while adding the file containing any secrets to `.gitignore`. If a key has already been pushed, it should be revoked and rotated immediately, since removing it from the latest commit alone does not erase it from the repository's history.

Q30. In your own words, summarize the complete journey of an API call — from client request to server response — using the concepts covered in this session.

It starts with the client — an application or script — deciding it needs some data or wants an action performed, so it builds an HTTP request made up of a URL, a method (GET, POST, etc.), headers (including authentication like a Bearer token), and sometimes a body containing JSON data. This request is broken into packets and sent across the internet, first resolving the domain name to an IP address through DNS, then routed through the network to the destination server. The server receives the request, checks the authentication headers to confirm the client is authorized, processes the request, and prepares a response, which usually comes back as JSON along with a status code such as 200 for success or 4xx/5xx for various errors. The client then reads this response — checking `response.status_code` to confirm success and using `response.json()` to extract the actual data — and finally uses that data within the application. Throughout this entire journey, following best practices like setting timeouts, handling exceptions, and keeping API keys secure is what keeps the process reliable and safe.